

UserLand Frontier

The UserTalk Language

This chapter provides an overview of the UserTalk scripting language.

Repetition, Decisions

UserTalk includes many language "structures" for repetition (looping) and decisions (branching). Each structure consists of one or more "blocks", i.e. a keyword line and one or more lines indented underneath (i.e. at a lower level). A keyword line contains one or more reserved words (such as "for" or "if") and usually user variables, data or expressions.

Note that these examples are just script fragments; they may assume that certain local variables have been defined or specific verbs exist earlier in the script (e.g. to open a file). As you review the examples, remember that most Frontier verbs follow the category.name format. Verbs will be covered in detail later in this chapter.

A **for** loop repeats a specified number of times, incrementing a variable each time.

```
for i = 1 to 5
    msg (i)
    clock.waitSeconds (1)
```

A **for** loop can also iterate through every element of a list, assigning the loop variable to each element in turn.

```
for element in finderMenu.getSelectionList ()
    dialog.alert (element)
```

A **while** loop repeats as long as a certain condition is true.

```
while not file.endOfFile (f)
    msg (file.readLine (f))
```

A **fileloop** structure iterates through every item in a folder or on a disk (or even every disk), assigning the loop variable to each element in turn.

```
fileloop (f in path)
    msg (file.fileFromPath (f))
```

You can include a "depth" parameter in the fileloop statement, indicating the number of levels of sub-folders that are traversed:

```
fileloop (f in path, infinity) //visit all files at all levels
    msg (file.fileFromPath (f))
```

A **loop** structure repeats forever (sort of). To exit from a loop, use "break". Although a loop and break sometimes looks easier, many software experts recommend rewriting your script to use "while" or "for" instead of "loop". Frontier leaves the choice to you.

```
loop
    msg (i)
    i = i + 1
    if i > 5
        break
```

An **if** decision structure executes the block of one or more indented lines if the expression is true.

```
if answer > 6
    msg ("Big family!")
```

An **if** structure may include an else block. It will be executed if the expression is false.

```
if answer > 6
    msg ("Big family!")
else
    msg ("Not such a big family!")
```

A **case** structure executes the block of code that corresponds to the value of the expression. The else block is optional. Note that more than one value (or expression) may lead to the same action.

```
case letter
    "a"
    "e"
        msg ("Vowel")
    "b"
        msg ("Consonant")
else
    msg ("This example isn't very thorough.")
```

Some languages include an **else if** extension to the standard **if** structure. Frontier's case statement can be used to create an equivalent.

```
case true
    temperature < minimum
        dialog.alert ("Too cold.")
    temperature > maximum
        dialog.alert ("Too hot.")
else
    dialog.notify ("Delicious porridge.")
```

Error Handling

The first way to handle errors is to avoid them! Frontier includes many verbs that can be used to check possible error-causing situations before the error arises. For example, before you call `file.delete`, you can call `file.exists` to ensure you are not attempting to delete a file that isn't there.

```
if file.exists (f)
    file.delete (f)
```

There are two sorts of errors that a running script may encounter: some verbs return "false" if they fail, others halt the script. For verbs that return true or false, the "if" statement is typically used to catch errors (often in conjunction with the "not" keyword).

```
if file.open (f)
    theLine = file.readLine (f)
```

In the previous example, the sole purpose of the "if" line was to check a certain condition -- it didn't perform any action (only the indented line performed an action).

To handle errors that normally halt the script, UserTalk includes a **try** structure.

```
try
    file.delete (f)
```

The try block will trap the error and the script will continue (assuming there are additional statements after this small fragment). As with any other block structure in Frontier, a try block can contain multiple statements (lines). If any line causes an error, the remaining lines in the block will be skipped. A try block has an optional **else** block, providing code to execute when an error is encountered. The else block even has access to the error message, through a

variable called "tryError". Sometimes you just want to display the error and continue.

```
try
    file.delete (f)
else
    dialog.alert (tryError)
```

Simple scripts or those used only occasionally may not need error-handling. When you create scripts that you use regularly -- or to share with others -- think about different ways the script could fail and use the techniques described in this section to anticipate and handle possible errors.

Bundles

The ability to collapse sections of code to hide unnecessary detail is a key advantage of editing scripts as outlines. UserTalk's looping, branching and error-handling blocks all have an outline structure. Sometimes it's useful to group other lines together. Bundling a group of related lines can result in code that is more readable at a higher level than the individual lines would be. Sometimes it's a good idea to bundle lines that make up a multiple-line process whose details many or most readers of your script will not need to understand.

A **bundle** structure is used to group a series of lines into a single block.

```
bundle //create a SimpleText file
    file.new (path)
    file.setType (path, "TEXT")
    file.setCreator (path, "ttxxt")
```

There are two ways to create a bundle. The first is to type the word "bundle" into your Frontier script and then indent the group of lines under this heading. (An easy way to indent those lines is to choose the "Demote" option from the Outliner menu.) The second is to position the cursor on the first line you wish to be part of the bundle and then choose "Bundle-ize" from the Script menu. You can undo a bundling operation by positioning the cursor on the heading where the "bundle" is defined or any line contained in the bundle and choosing the "De-Bundle" option from the Script menu.

Variables

As with most programming or scripting languages, Frontier supports two types of variables: local and global. **Local variables** are those that occur temporarily within a script, local script or block and exist only within that context.

Although Frontier does not require that you specifically declare local variables, it's a very good idea and can save you hours of debugging that would otherwise be required to track certain kinds of errors. Declaring local variables is a good habit in general and particularly important in scripts that can be called from other UserTalk scripts.

A **local** structure is used to declare (define) one or more variables as local. A script may contain more than one local structure; all the variables are local to the block in which the declaration appears. (This context is often referred to as the "scope" of the variable.)

```
local
    x
    y
    z
```

You can also set the initial value of one or more variables within a local block, and include end-of-line comments:

```
local
    x = 3
    y = 4 //blah, blah, blah
    z
```

UserTalk often provides more than one way to accomplish the same result, accomodating different styles. With some extra punctuation, you can create locals all on one line:

```
local (x)
```

Setting the initial value is also possible:

```
local (x = 3)
```

Multiple variables are supported:

```
local (x, y)
```

One or more variables can also include values:

```
local (x = 3, y = 4)
```

Note that these alternate forms make "local" look somewhat like a verb; if you find that confusing, just use the indented form. Even if you choose one form for

your own scripts, be sure to learn to recognize the other form as both are quite common.

Global variables are those that are permanently stored in the Frontier Object Database and have cell addresses there. Frontier's global variables are also "persistent," i.e. they retain their value even after the script finishes. The value is still there when you open Frontier again. These "persistent" variables are completely separate from scripts that use them, so their value is not altered or destroyed by changing and "compiling" a script. Because they live in a table in the Object Database, you can view and edit them on their own, independent of any script. Persistent global variables are a major benefit of Frontier and its Object Database.

Frontier Verbs

Frontier's vocabulary consists of several hundred verbs. These verbs are identical to functions in conventional programming languages. Various computer languages call these elements by different names, including subroutines, procedures and handlers. We refer to them as verbs because they are most often used to direct Frontier or some other program with which Frontier is interacting to do something. Action is the province of verbs in human speech.

The most important verbs can be divided into 21 categories. (There are additional categories. Every verb is documented in [DocServer](#), described later in this chapter.)

Table 4-1. Verbs Summary

Category	Uses, Comments and Notes
basic	Handle numbers, datatypes, objects, etc.
clock	Set and get system clock values. Time-stamp documents. Pause execution.
date	Get and set date values.
dialog	Display and return results from dialog boxes.
editMenu	Emulate behavior of Frontier's Edit Menu, including fonts and styles

<u>file</u>	All file operations, including copy, delete, move, create, and rename. Looking at file and folder contents, read and write text files, deal with volumes, parse path names, lock and unlock files, handle Macintosh aliases, etc.
<u>fileMenu</u>	Emulate behavior of the Frontier File menu.
<u>Frontier</u>	Turn agents on and off, make Frontier the active application, and get information about Frontier
<u>kb</u>	Determine if the Command, Control, Option, and/or Shift key is down.
<u>launch</u>	Open applications, control panels, documents, and code resources.
<u>menu</u>	Manage and manipulate the Frontier menubar and menubar objects
<u>mouse</u>	Find out where the mouse is and if the button is being pressed.
<u>op</u>	Edit, navigate in, rearrange, expand, collapse, retrieve information from and otherwise manipulate Frontier outline objects.
<u>search</u>	Handle Frontier's internal find-and-replace operations
<u>speaker</u>	Define the sound the internal speaker will make and activate it.
<u>string</u>	Manage and manipulate string objects, including fields, word and sentences.
<u>system</u>	Deal with external applications and find out what version of the Operating System is in use.
<u>table</u>	Manipulate and interact with Frontier Object Database tables and their contents.
<u>target</u>	Determine the Frontier object that will receive the next action(s) taken in the environment.
<u>window</u>	Manage and manipulate Frontier windows.
<u>wp</u>	Handle word processing objects and their contents, including setting properties, controlling the selection, formatting the appearance of text and changing its content.

As discussed earlier in the manual, the full name of a verb is composed of its category, followed by a period, followed by the name of the verb. (When you want to run the verb, you must include parentheses, and parameters if required.) For example, [string.upper](#) operates on a string, converting it to upper case.

```
allCaps = string.upper ("Hello world!")
```

Every verb returns a value, either status (true or false to indicate success or failure) or some piece of information. Verbs can be "nested" one inside another, passing the value from the innermost script to the outer script:

```
dialog.alert (string.upper ("Hello world!"))
```

In many cases, it's useful to know both the status (true or false) and one or more other pieces of information. For example, you might want to ask someone a question by presenting a dialog. If they cancel (status is false) your script will take one action, if they enter information, it will do something else. As with many programming languages, Frontier supports two kinds of parameters:

1. passing a value (either a literal or the value of a variable or expression)
2. passing an address or reference to a variable (including a local variable or a global stored in the Object Database)

When it gets an address, the verb can directly modify the variable that exists at that address -- and is free to use the return value for status (true or false). The "@" operator converts a variable into an address.

```
if dialog.ask ("Who are you?", @scratchpad.who)
    dialog.notify("You are " + scratchpad.who)
else
    msg ("user cancelled")
```

Some verbs set multiple variables. For example, to get the position of a specific Frontier window:

```
window.getPosition ("examples", @horiz, @vert)
```

If the "examples" window is not open, the verb will return false (and leave the horiz and vert variables unchanged). Note that horiz and vert should be declared as local variables prior to the above.

Whether a verb should return status or a value depends partly on how the verb is likely to be used, and partly on the preferences of the script writer. The advantage of returning a value is that the verb can be nested inside other verbs.

If you prefer this approach, use it for scripts that you write. How? When there is only one key piece of information (in addition to status), simply return that information. If there is more than one key piece of information to return, create a list (or possibly a record) and return that. Since you can no longer return false to indicate lack of success, use:

```
scriptError ("optional error message here")
```

The script that calls your script can trap the error in a "try" statement, preventing the script from halting.

DocServer

DocServer is a website that documents every Frontier verb, operator and structure keyword (e.g. "if", "try").

You can find it here: <http://docserver.userland.com/>

Let's lookup the "dialog.ask" verb that was used in the previous section. Go to the DocServer home page (above). Then scroll down to Dialog, which is under Frontier Environment Verbs. Click on Dialog to see a list of all the verbs in that category. Now you're here:

<http://docserver.userland.com/dialog/>

Look for "dialog.ask" on that page in the list of verbs. Click on it, then you're here:

<http://docserver.userland.com/dialog/ask>

As you can see from the above URL, you can construct the URL for most verbs like this:

`http://docserver.userland.com/ + verb category + / + verb name`

Each DocServer verb page includes the verb Syntax, Parameters, a description of the Action, what the verb Returns, Errors if any, one or more Examples, plus Notes and See Also. The See Also items are links, so you can easily navigate to related verbs.

Operators and special characters

UserTalk generally supports the same operators as most modern programming languages, plus a few extras. To accomodate a variety of preferences, Frontier

has word equivalents for many of the symbols. For example, many script writers may prefer "equals" rather than "==" which is easily confused with the "=" assignment operator. Example:

```
if x equals 10
    x = 0
```

Table 4-2. UserTalk Operators

Operator	Purpose
=	Assignment
+	Add numeric values or concatenate (join) string and character values
-	Subtract numeric values or remove the first occurrence of one string from another
*	Multiplication
/	Division (integer or real, depending on the datatypes)
%	Modulus (remainder of integer division)
== equals	Test for equality
!= notEquals	Test for inequality
< lessThan	Less-than comparison
<=	Less-than-or-equal comparison
> greaterThan	Greater-than comparison
>=	Greater-than-or-equal comparison
beginsWith	Compare one string to the beginning of another
contains	True if one string contains another
endsWith	Compare one string to the end of another

 or	Logical OR
&& and	Logical AND
!	Logical NOT
++	Increment
--	Decrement
@	Address-of
^	Dereference an address

Although they aren't operators, there are a few other characters that have a particular meaning in UserTalk.

Table 4-3. Other Special Characters

Character	Purpose
"abcdef"	Define a string (with straight quotes). The string may include curly quotes. To include a straight quote, precede with a backslash.
'a'	Define a character with single quotes. To include a single quote, precede with a backslash.
'abcd'	Define a "string4." Some Mac OS functions require this type.
[...]	Evaluate a variable or expression. Also used for lists, records and the object model.
["..."]	Include an object database name that contains spaces or other reserved punctuation.
	Treat the next character as a literal. (e.g. to include a quote in a quoted string)
t	Tab character (in a quoted string)
r	Return (cr) character (in a quoted string)
n	Linefeed (lf) or "newline" character (in a quoted string)

Frontier Datatypes

UserTalk is a rich language that is capable of dealing with 28 different types of data. Many of these datatypes are useful only to advanced script writers. The primary datatypes are summarized below. Note that characters and strings support the full 255-character set.

Table 4-4. Common UserTalk Datatypes

Datatype	Range of Legal Values
addressType	Any Object Database cell or any non-existent cell in an existing table
booleanType	True or False
charType	Any character, enclosed in single quotes
dateType	Any legal system date value
directionType	up, down, left, right, flatup, flatdown, nodirection
intType	-32768 to 32767
longType	-2147483648 to 2147483647
menubarType	Any Frontier menubar object
outlineType	Any Frontier outline object
scriptType	Any Frontier script
stringType	One or more characters, enclosed in double quotes
string4Type	Exactly four characters, enclosed in single quotes
tableType	Any Frontier Object Database table
wptextType	Any Frontier word processing text object

This is a [Manila](#) site.